



University of Pisa

Department of Information Engineering

Master's Degree in Artificial Intelligence and Data Engineering

Smart Health Monitoring System

Student: Antonio Pepe
Registration number: 665417

Teachers: Prof. Giuseppe Anastasi
Professor Carlo Vallati
Engineer Francesca Righetti

Academic Year 2025/2026

Contents

1 Introduction 1.1	2
Application domain	2
2 Architectural diagram	3
2.1 Network Protocol	3
2.2 Data encoding	4
2.3 Configuring Devices	4
3 Machine Learning Model 3.1	5
Main Function	5
3.2 Datasets	5
3.3 Training	5
3.3.1 Model Accuracy	6
3.3.2 Device Inference	6
4 Cloud Application 4.1	7
Backend	7
4.2 Closed-Loop	7
4.3 User Interface	7
4.4 Hardware Interactions	8
4.4.1 Button: Activation of the Adaptive Mechanism.	8
4.4.2 LED: Patient Status Indicators	8
5 Evaluation and Stress Testing	9
5.1 Node Failure Scenario	9
5.2 Adaptive Mechanism	9
5.2.1 Activation via Hardware Button	9
5.2.2 Activation via User Interface	9
5.3 Grafana	10
5.3.1 Normal and stressed conditions	10
5.3.2 Grafana Queries	10

Chapter 1

Introduction

1.1 Application domain

The application domain chosen for this project was "SmartHealth." The project involves a patient monitoring system. The goal was to measure values such as body temperature, heart rate, and oxygen saturation through sensors in order to predict dangerous or emergency situations and, via an actuator, notify a potential expert of the patient's condition.

My project was based on some papers I found on Google Scholar:

1. **The Internet of Things for Health Care: A Comprehensive Survey** (SMR Islam, D. Kwak, M. H. Kabir, M. Hossain and KS Kwak — *IEEE Access*, 2015)

It provides an overview of the sampling requirements of biomedical sensors and formalizes the three-tier network architectures from Sensor → Gateway → Cloud.

2. **Self-Aware Early Warning Score System for IoT-Based Personalized Healthcare** (I. Azimi, A. Anzanpour, M. Rahmani-Andebili, et al. — *FedCSIS*, 2016)

This work describes the digitization and automation of EWS (*Early Warning Score*) and MEWS clinical protocols within IoT ecosystems. It explains how to map the combined data of temperature, heart rate, and oxygen saturation (2) into numerical risk classes. This study forms the logical basis of the machine learning algorithm.

Chapter 2

Architectural diagram

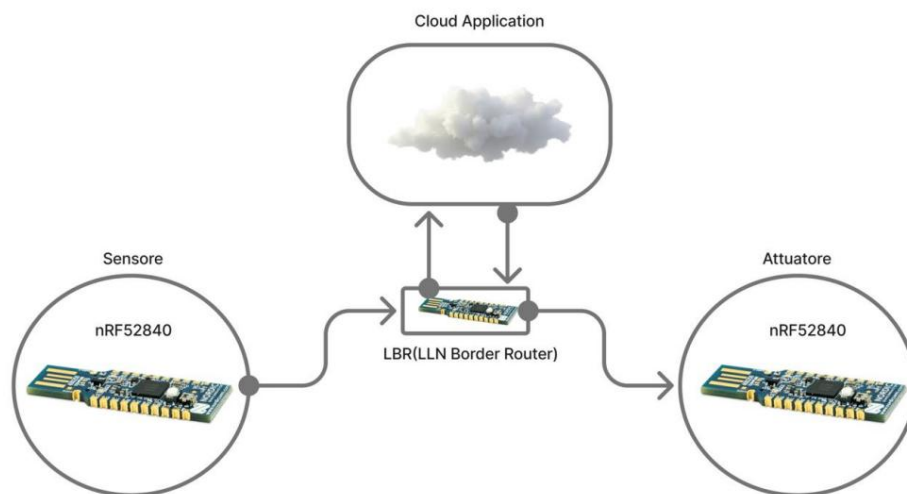


Figure 2.1: Architectural diagram of the IoT application.

The application schematic is very simple. We have a Border Router implemented with an nRF52840. Attached to the BR are two physical nRF52840 dongles, one acting as a sensor and one as an actuator. The flow of operations follows the arrows in the figure.

2.1 Network Protocol

As a network protocol, I decided to use CoAP (Constrained Application Protocol). It's a UDP-based application-layer protocol, making it lighter, but its drawback is that it's unreliable. In reality, for the situation we're in—designing IoT networks, which involve devices with limited resources—this protocol is perfect. Furthermore, the protocol offers various methods and functions; in fact, it's valid for both the client and server sides.

Finally, CoAP, by definition, is a protocol that makes resource observing available: in our case we have sensors/actuators that expose resources that the Cloud Application needs to change.

2.2 Data encoding

The encoding method I used was JSON (JavaScript Object Notation). I opted for this solution because JSON is one of the most common formats used, and it doesn't have all the rules of an XML (Extended Markup Language) file. JSON files are also simple, with a restricted syntax for objects and arrays. In my project, for example, as you can see in the `res_vitals.c` file, I used:

```
int length = snprintf ((char *) buffer , preferred_size ,
    "{" hr \": %d, \" body_temperature \": %d, \" spo2 \": %d, \" risk \": %d}", current_hr , current_temp , current_spo2 ,
    current_risk );
```

Listing 2.1: Building the JSON payload in `res_vitals.c`

2.3 Configuring devices

Configuring the devices was the relatively more complex part. I configured the nodes as explained in the architectural system section. The nodes were configured statically using a Python file (`configuration_manager.py`):

```
# configuration_manager.py DB_CONFIG = { #
configurations for DB connection "host": " localhost ", "user": "root", "password": "1", "database": "
    SmartHealthIoT "

}

SENSORS_CONFIG = {
    "Sensors": { "id": 1, "
        sensor_ip ":
            "fd00 :: f6ce :36 b9:a760:ecea", # IP of the dongle that acts as
                sensor
            " actuator_ip ": "fd00 :: f6ce :36 a6:c68f:cb04" # Physical actuator IP
        }
    }
```

Listing 2.2: Database and Sensor Configuration in `configuration_manager.py`

As you can see in this code excerpt, in addition to the static configuration of the devices there is also the configuration of the credentials for the DB.

Chapter 3

Machine Learning Model

3.1 Main function

A machine learning model, specifically Random Forest, was used to generate multiple decision trees based on vital signs from a dataset. The model collected data regarding temperature, heart rate, and oxygenation. Thanks to these three parameters, I was able to train the model and associate a risk class with each situation. As per design, tinyML was used to infer the model from the sensor.

3.2 Dataset

The dataset used to train the machine learning model was found on Kaggle. Specifically, the following dataset was used:

<https://www.kaggle.com/datasets/jacobhealth/health-status-dataset>

The dataset describes the relationship between body temperature, heart rate, and oxygen saturation in the blood (SpO2). The patient's health status is represented by a numerical label ranging from 0 to 2, according to the following classification:

- **0** – No abnormal parameters detected;
- **1** – The patient is ill, but the condition is not life-threatening;
- **2** – The patient is in danger of dying.

In the dataset we find more than 5000 tuples.

3.3 Training

The model was trained using Google Colab, as documented in the ML_model.ipynb notebook located in the Machine_Learning/ directory of the project.

The procedure followed five basic steps.

1. **First step:** uploading the dataset and updating libraries.
2. **Second step:** preprocessing. The three available features (temperature, heart rate, and SpO2) were selected as input to the model, while the target column was used as a label for classification.

3. **Third step:** model selection. A Random Forest algorithm was chosen, a classifier which combines multiple decision trees to obtain reliable predictions. The choice fell on this algorithm because it is particularly suitable for execution on embedded devices with limited resources, thanks to the possibility of being converted into efficient C code via the library `emlearn`.
4. **Step 4:** Training and validation using Google Colab.
5. **Fifth step:** Export for embedded deployment. The trained model was converted to C code using the `emlearn` library. The generated file, `vitals_classifier.h`, contains the `vitals_rf_model_predict()` function which is then included directly in the firmware of the sensor node.

```
#3. RANDOM FOREST TRAINING
# (Optimized for Contiki -NG)
print("\nTraining the Random Forest model on real data from Kaggle ...")
# I keep 5 trees and depth 3 to avoid saturation
# the limited RAM memory of nodes in Cooja
random_forest_model = RandomForestClassifier (
    n_estimators =5, max_depth =3, random_state =42
)
random_forest_model .fit(X_train , y_train)

#4. ACCURACY EVALUATION
y_pred = random_forest_model .predict(X_test)
print("\n--- TINYML ACCURACY REPORT ---")
print( classification_report (y_test , y_pred))
print(f"Risk levels calculated by the model: "
      f"{ random_forest_model .classes}")
```

Listing 3.1: Training the Random Forest Model in `ML_model.ipynb`

3.3.1 Model Accuracy

After training, the Random Forest model was evaluated on the test set (20% of the data, corresponding to 1182 samples). The results obtained are shown below:

--- TINYML ACCURACY REPORT ---

	precision	1.00	recall	f1-score	support
0			1.00	1.00	215
1	1.00		0.90	0.95	473
2	0.91		1.00	0.95	494
accuracy				0.96	1182
macro avg	0.97		0.97	0.97	1182
weighted avg	0.96		0.96	0.96	1182

As the report shows, the model achieves an overall accuracy of 96%.

3.3.2 Device Inference

The `vitals_classifier.h` file is included in the sensor node code, specifically in the `sensor.c` file. At each sampling cycle, the sensor performs model inference locally. The three parameters `vitals` are inserted into a feature array and passed to the prediction function, which returns the clinical risk level. This value is then exposed via the CoAP `/vitals` resource and sent to the cloud backend.

Chapter 4

Cloud Application

4.1 Backend

Both the backend and frontend are written in Python. I preferred to use a virtual environment (venv) to isolate all dependencies. The cloud system is located in the `Cloud_Application/` folder. Within this folder, we find two files related to the backend application:

- `db_setup.sql`: This SQL script creates the SmartHealthIoT database and the Health_Measurements table, defining the columns to store vital data (heart rate, temperature, SpO2, clinical risk, sensor status and timestamp) from the IoT nodes.
- `SmartHealthCloud.py`: This Python backend uses CoAP's *Observe* mechanism to asynchronously receive sensor data without having to periodically query it. When the JSON data arrives, it parses it, saves the measurements to the MySQL database, and, if the clinical risk level has changed, sends a command to the actuator to update the alarm status (green, yellow, red).

4.2 Closed Loop

The system I developed works like this: the sensor collects data relating to the vital parameters mentioned above (in reality some parameters are generated randomly, as well as a gap added to the temperature because the sensor simulates its internal temperature).

The sensor then uses the Random Forest model to perform inference and determine the clinical risk class from the collected data, thus implementing an *edge computing approach*. The sensor then sends the risk class and vital data via the CoAP protocol to the Cloud Application.

The backend system, implemented in the `SmartHealthCloud.py` file, receives data via CoAP *Observe*, saves it to the MySQL database, and implements the decision-making logic that determines whether and how to act on the actuator. As soon as a change in risk class is detected, the actuator is contacted, again via CoAP, by the Cloud Application to change the LED color.

4.3 User Interface

The user interface was developed using the Python Tkinter library. To launch the interface, activate the venv environment, navigate to the `Cloud_Application/frontend/` directory, and run the following command:

```
python SmartHealthUI.py
```


Once this is done, a panel will open displaying all the parameters currently measured by the sensor and the risk class inferred directly from the sensor node. You can also control the update *rate* .

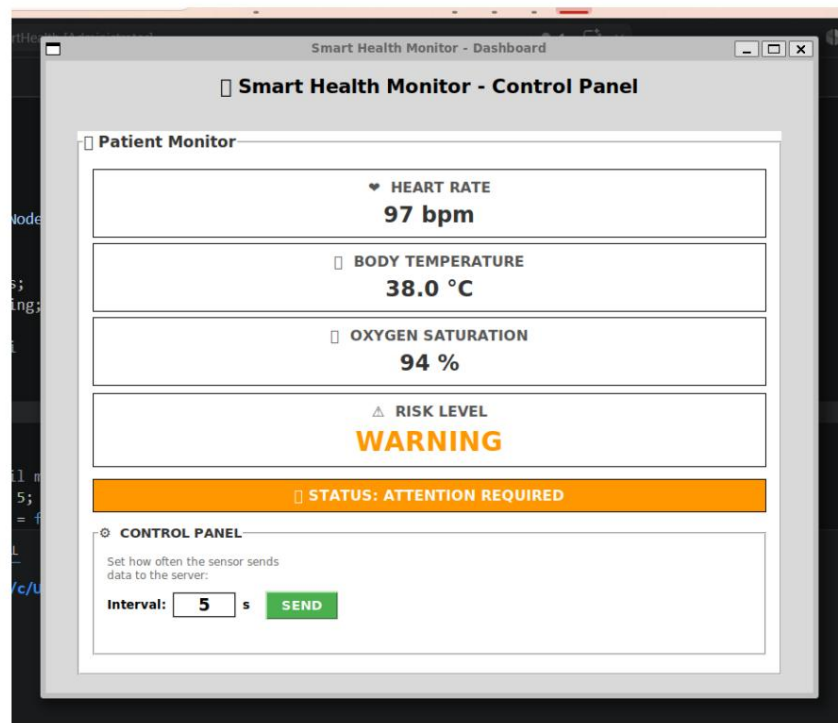


Figure 4.1: UI interface screenshot for SmartHealth

4.4 Hardware Interactions

The SmartHealth system uses the buttons and LEDs available on the IoT nodes of the dongles.

4.4.1 Button: Activation of the Adaptive Mechanism

The hardware button on the sensor node has been configured to activate the adaptive mechanism for responding to network stress conditions. The logic is implemented in the sensor's main process (sensor.c file). When the adaptive mechanism is active, the sampling period is increased from 5 to 20 seconds. Pressing the button again returns the system to normal operating conditions.

4.4.2 LED: Patient Status Indicators

The project also uses LEDs. As explained in the closed-loop section, whenever a change in risk class is detected, the cloud application notifies the actuator via CoAP.

- **Risk 0 (normal):** Green LED, stable situation.
- **Risk 1 (warning):** yellow LED, the patient is ill but not in danger of dying.
- **Risk 2 (emergency):** Red LED, the patient is in danger of dying.

Chapter 5

Evaluation and stress test

5.1 Node Failure Scenario

The actual sensor, connected via USB to the PC and flashed with the sensor.c firmware, can be physically disconnected. When this happens, the node disappears from the RPL network and the cloud backend no longer receives CoAP notifications. The system detects the failure through the DB-Manager's `fetch_latest_measurements()` method, which periodically checks the dynamic timeout based on the sampling rate. When the time elapsed since the last data point exceeds the timeout, a record with the `SENSOR_MISSING` status is created. A check is performed in the user interface: when the sensor is unreachable, all dashboard elements are disabled. At the database level, the `Health_Measurements` table includes a status column that can assume the value `ONLINE` or `SENSOR_MISSING`. When a sensor stops sending data for a certain period, the system updates this field to indicate the failure, allowing the dashboard to display the node's status.

5.2 Adaptive Mechanism

The system implements an adaptive mechanism that allows the sensor sampling frequency to be dynamically modified in response to network stress conditions. This mechanism can be activated through two complementary methods: a hardware button on the sensor node and a user interface (UI).

5.2.1 Activation via Hardware Button

As described in Section 4.4.1, the physical button on the sensor dongle allows you to toggle the state of the adaptive mechanism. Pressing the button toggles the sampling period between 5 seconds (normal mode) and 20 seconds (adaptive mode).

5.2.2 Activation via User Interface

In addition to hardware control, a feature has been implemented directly in the graphical interface (`SmartHealthUI.py` file) that allows simulating network stress conditions by varying the sensor *sampling rate* in real time.

There is a dedicated control panel in the UI that exposes a numeric input field.

The user can enter a value (expressed in seconds) representing the new desired sampling interval. The cloud backend, via CoAP, sends this value to the sensor node, which dynamically updates its data acquisition rate.

On the sensor side, a dedicated CoAP resource `/sampling` has been implemented that accepts PUT requests to change the `current_sampling_rate` parameter at runtime.

5.3 Grafana

5.3.1 Normal and stressed conditions

Under normal conditions, the sensor samples and sends data every 5 seconds. The graphs show a regular and continuous flow. This is the basic behavior of the system, what you would expect in an uncongested network.

When you press the button on the sensor, the adaptive mechanism activates and the sampling period increases from 5 to 20 seconds. This change is immediately visible on the dashboard: the points on the graph become sparser, because the data arrives less frequently.

Furthermore, since the sampling rate can be changed even at intervals of less than 5 seconds, the graphs show a higher and more closely spaced density of points, thus highlighting a higher network load and greater system stress.



Figure 5.1: Grafana Graphs

5.3.2 Grafana Queries

Query 1 - Vital Signs Trends

This query extracts and aggregates the three measured physiological parameters (heart rate, body temperature, and oxygen saturation) over time, calculating their average.

Query 2 - Clinical Risk Level

Returns the time evolution of the risk class.

Query 3 - Packets Received per Minute

Counts the number of CoAP messages received by the cloud application in one-minute windows. This metric quantifies the impact of the adaptive mechanism.

Query 4 - Sensor State Over Time

Monitor the IoT node by distinguishing between ONLINE state (normal operation) and SENSOR_MISSING state (node unreachable).